

基于蒙特卡洛思想的多重生产决策问题

摘要

生产生活中有大量决策问题，为了追求利润、成本等目标，人们往往需要进行复杂的思考过程。针对这一问题和具体的题目要求，本文给出了一个契合检验、复杂实际模拟与简化模型评估结合的量化决策模型，来辅助决策。

对于问题一，本文在序贯概率比检验的基础上进行简化，得到了使用抽样检测验证产品次品率是否符合标称值的方案选择模型。本文使用 Z 检验的施行特征函数，从限制犯第 II 类错误的概率出发，反向获得了检测所需的最少的样本容量 n ，进而得到临界次品数量 k 。

对于问题二，本文基于蒙特卡洛法的思想，进行模拟实验以选择优化的决策方案。通过大容量样本的随机采样，其结果可以逼近真实结果。基于该出发点，本文设计了蒙特卡洛优化实验的模拟算法。在不同的情况下，算法复刻生产线的全部决策过程，在进行不同决策方案的模拟生产运行时，将一定量的零配件投入，会通过检验或使用次数判断丢弃其中部分零配件，在用尽一种零配件无法继续生产后，统计售出数目和总成本计算纯利润。比较各决策方案的利润可得到对应情况的最优决策方案和最大预期利润。另外，本文还对结果进行了定性分析，以验证模型的结果。

对于问题三，本文在第二问的基础上完成了复杂化的实际问题分析，并给出了简化模型思路作为解决多零配件、多工序的复杂决策优化问题的一般方法。通过减少投入的零配件数、拆解子决策、人为定性判断，均可大幅减少模型的运算量，而且可以在定性的角度上保持算法结果的正确性。

对于问题四，本文在前三问的基础上进行扩展，得到了在次品率标称值未知的情况下，解决多重决策问题的模型。本文首先针对次品率未知的情况，应用贝叶斯推断，得到了检验任何样本所需的最少次数模型，其能在指定可信度下给出相应的样本容量和次品率。然后将贝叶斯推断得到的 beta 分布带入生产模拟的过程中，实时更新，达到了与问题二、三中模型相同的决策能力水平。

关键字： 多重决策 简化序贯概率比检验 蒙特卡洛优化模拟实验 贝叶斯推断

一、问题重述

1.1 问题背景

在工厂生产、商业决策等一系列社会生产生活中，往往面临着多重决策问题。每个决策问题有多个选择，不同选择会对接下来的决策流程产生影响，进而对整个决策问题的最终效益评判产生影响，这使得寻找整个问题的全局最优解成为难题。在面对参数变化的选择情况时，问题进一步复杂化。本问的零配件检测、组装、拆解问题，正是这一问题的典例。寻求这样一个问题的定量评估模型，有着重要的意义和广阔的应用前景。

1.2 相关研究

对前后步骤相互影响的多重决策问题，根据具体的题设情况，可以有多种解决方案，以下简要列举几种方法：

1. 动态规划：关于可以划分为诸多有类似结构子问题的决策问题，动态规划有着较强的针对性。如左正康等人验证了命令式动态规划类算法，其适用于子问题之间存在大量依赖关系和约束条件的情况 [?];
2. 决策树：决策树作为解决优化问题的重要方法，得到了深入研究。如刘小虎等人早年前改进的算法，其在选择新属性时，兼顾该选择带来的信息增益和选择后其他选择带来的信息增益 [?];
3. 进化算法：遗传算法、粒子群算法等进化算法也适用于优化决策问题。如连可等人用遗传算法优化了支持向量机的决策树 [?].

1.3 问题要求

本问题的四个子问题是递进关系，在生产决策的问题复杂化的过程中，求取不同生产情况所适用的检测方案。

1. 问题一要求在一定的标称值下，对零配件检测的两种情形，提供适宜的检测方案，以使检测次数最少，来决定是否接受该批零配件；
2. 问题二针对零配件组装成品问题的多种情形，要求给出零配件检测、成品检测、拆解决策、次品调换的整体决策方案，并给出相应的依据和指标；
3. 问题三将问题升级到多道程序，同样探求决策方案以及相应的依据和指标；
4. 问题四将次品率的获取纳入到检验问题中去，使问题更加现实化，得出的决策方案更加真实。

二、问题分析

2.1 问题一

针对问题一，由于样品检测过程相互独立，根据给定的次品率标称值 $p_0 = 10\%$ ，我们可以将每一次抽样检测视为 (0-1) 分布，并将事件的发生按如下规律编码：

$$\begin{cases} X = 0, \text{该零件为良品} \\ X = 1, \text{该零件为次品} \end{cases}$$

显然，一批零配件的抽样检测可视为 n 重伯努利试验，其随机变量服从 $b(n, p)$ ，其中 p 为该批样品的实际次品率。由序贯概率比检验的思想出发，基于中心极限定理和正态分布的 0-1 分布参数的区间估计，通过构造一个关于样本容量 N 等多个统计量的可迭代方程，可求解出一个稳定的且符合置信水平的样本容量。此外，利用 n 重伯努利分布的分布函数，可以确定对应样本容量下的临界次品数。最终给出检测次数尽可能少的抽样检测方案。

2.2 问题二

针对问题二，由于各部分决策相互影响，且生产中存在返厂拆解的迭代过程，很难用一个数学表达式来准确描述受决策选择影响的利润，这使得依据函数评价的方法失效。应用动态分析、遗传算法等方法会大大增加问题描述的复杂程度，使原本清晰的决策问题复杂化。因此，本文倾向于采用简单直观的方式。考虑蒙特卡洛算法的近似解在样本足够多的情况下可以表示问题的真实解，本文拟进行大批量样本的生产模拟实验，在实际的流水线中计算一批零配件所能产生的最大效益。

2.3 问题三

问题三作为问题二的复杂化、公式化，是对模型的可拓展性的考验。鉴于问题二模型简单直观的特性，其拓展到一般化问题中将具备同样优秀的结果。为了保证计算量在决策步骤增多的情况下不几何增长，可以考虑在保证模型结果准确度的条件下，对模型进行适当简化，即不影响决策方案的选择，弱化直接估计利润低能力。

2.4 问题四

针对问题四，由于在生产时未知物品次品率的标称值，需要将该值的获取纳入到模型中去。由于在实际的模拟实验中，可以获得先验概率和由观测数据的统计模型得出的似然函数，考虑使用贝叶斯推断的方法，计算后验概率，在生产过程中实时进行修正。

三、模型假设

为了对模型进行合理简化，小组建立了如下的模型假设：

1. 在问题一中，零配件的实际次品率会在标称值 0.1 附近，且最大不超过 0.5。为应用中心极限定理，抽样检测次数不宜过少，大于 20 较为适宜；
2. 每个检测对象相互独立，服从二项分布（或正态分布）；
3. 由于问题本身已经有较高的复杂度，判断决策的标准仅选择了最终的纯利润。对诸如一次生产线模拟所耗时间（进行的各种操作次数）、出售次品带来的非经济层面的负面影响等，不再加以考虑。

四、符号说明

符号	含义	单位
n	样本容量	个
k	临界次品数量	个
p_0	次品率标称值	%
α	置信水平	%
δ	实际次品率与标称值的容忍误差	1
$index$	决策方案的索引，由二进制编码转化为十进制获得	1
$costpart_i$	零配件 i 的购买单价	元
$costassembly$	装配成本	元
$priceproduct$	市场售价	元
$defectivepart_i$	零配件 i 的次品率	%
$costcheckpart_i$	零配件 i 的检测成本	元
$defectiveproduct$	成品次品率	%
$costcheckproduct$	成品检测成本	元
$lossreplace$	调换损失	元
$costdisassemble$	拆解费用	元
$numparts$	模拟中第 i 个零配件的个数	个
$numgood_i$	模拟中第 i 种零配件的良品个数	个
$numbad_i$	模拟中第 i 种零配件的次品个数	个
$usespart_{ij}$	模拟中第 i 种零配件的第 j 个的使用次数	次
$maxuses$	零配件最多使用次数	次
$totalsales$	成交数	个
$cost$	成本	元

符号	含义	单位
<i>profits</i>	利润	元
<i>bestprofit</i>	最大利润	元

五、模型的建立与求解

5.1 问题一 简化的序贯概率比检验

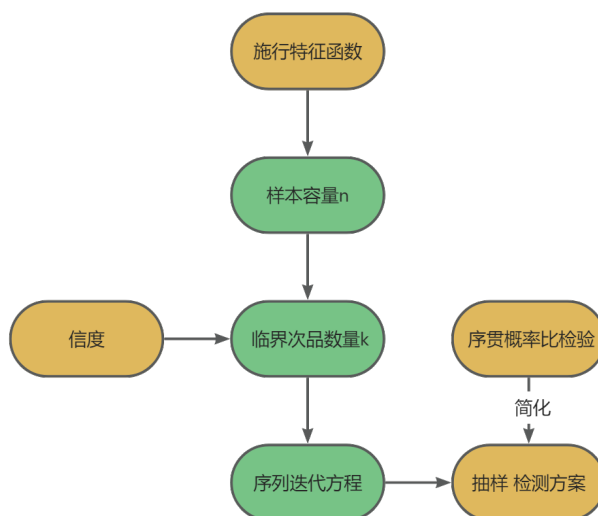


图 1 问题一流程图

序贯概率比检验（Sequential Probability Ratio Test, SPRT）是一种统计决策方法，用于在连续收集数据的过程中，以最小的样本量尽快地做出决策，其主要用于二项分布或正态分布的假设检验中。本问中对该方法进行简化，以求解检测零配件所需的最少数目。

5.1.1 理论基础——概率论及统计原理

1. (0-1) 分布与二项分布

(a) (0-1) 分布（伯努利分布，Bernoulli Distribution）

设随机变量 X 只可能取 0 与 1 两个值，它的分布律是

$$P\{X = k\} = p^k(1 - p)^{1-k}, k = 0, 1 \quad (0 < p < 1)$$

对于一个随机试验，如果它的样本空间中只包含两个元素，即

$$S = \{e_1, e_1\},$$

我们总能在 S 上定义一个服从 (0-1) 分布的随机变量

$$X = X(e) = \begin{cases} 0, & \text{当 } e = e_1 \\ 1, & \text{当 } e = e_2 \end{cases},$$

来描述这个随机试验的结果。

(b) 伯努利试验 (Bernoulli Trial)

伯努利试验 E 只有两个可能的结果: A 及 $\bar{A}$ 。设 $P(A) = p (0 < p < 1)$, 此时 $P(\bar{A}) = 1 - p$ 。将 E 独立重复地进行 n 次, 这一串试验称为 n 重伯努利试验。下图??展示了二项分布及其累积分布的具体情况。

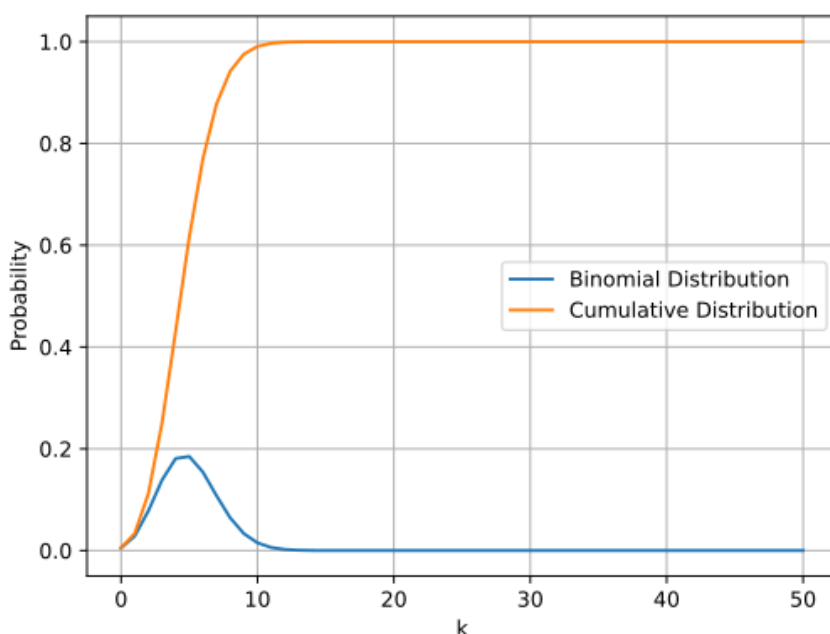


图 2 二项分布及其累积分布

(c) 二项分布 (Binomial Distribution)

二项分布是一种离散概率分布, 其描述了固定次数的独立伯努利试验中成功次数的概率分布。以 X 表示 n 重伯努利试验中事件 A 发生的次数, 其是一个随机变量, 分布率为

$$P\{X = k\} = \binom{n}{k} p^k (1-p)^{n-k} \geq 0, k = 0, 1, 2, \dots, n.$$

X 取 0 到 n 所有次数的可能之和为 1, 即

$$\sum_{k=0}^n P\{X = k\} = \sum_{k=0}^n \binom{n}{k} p^k (1-p)^{n-k} = (p + 1 - p)^n = 1.$$

(d) 累积分布函数 (CDF)

累积分布函数表示一个随机变量小于或等于某个给定值的概率。在二项分布中, 累积分布函数 $F(k, n, p)$ 给出了在 n 次试验中成功次数小于或等于 k 的概率, 其

是二项分布概率质量函数（PMF）的累积和：

$$F(k, n, p) = \sum_{i=0}^k \binom{n}{i} p^i (1-p)^{n-i}.$$

2. 抽样检验与置信水平

- (a) 抽样检验：在质量控制中，抽样检验是用来评估一批产品的质量是否符合预定标准的方法。通常设定一个可接受的次品率 p ，然后从批量中抽取一个样本，检查样本中的次品数。如果样本中的次品数超过了某个预先设定的临界值 k ，则认为整个批次不合格。
- (b) 置信水平：对检验结果正确性的信任程度。
- (c) 临界次品数量 k 的实际意义：在给定的样本容量 n 和置信水平 α （如 0.05 或 0.10）下，次品数可以容忍的最大界限。如果实际次品数超过了 k ，即可拒绝次品率不超过标称值的假设。如下图??所示。

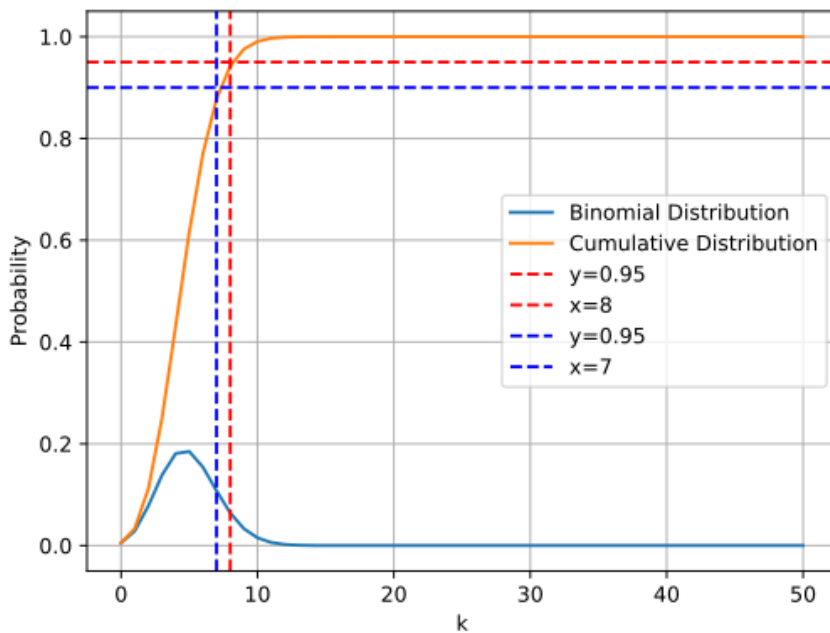


图 3 基于分布函数的临界次品数目

在抽样检测中， k 值帮助我们确定何时停止检测，因为一旦次品数达到或超过 k ，我们就可以做出决策。

3. 施行特征函数 (OC 函数)

在进行假设检验时，通过显著性水平控制犯第 I 类错误的概率，而犯第 II 类错误的概率往往通过样本容量来加以控制，因此可以通过限制犯第 II 类错误的概率来反向选取样本容量。由于样本容量需要在几百乃至上千的数量级，所以选择 Z 检验而非 t 检验。以下由 Z 检验右边检验问题的 OC 函数来求取样本容量：

$$H_0 : \mu \leq \mu_0 = 0.1, H_1 : \mu > \mu_0 = 0.1 ,$$

其 OC 函数是:

$$\begin{aligned}\beta(\mu) &= P_{\mu}(\text{接受 } H_0) \\ &= P_{\mu}\left\{\frac{\bar{X} - \mu_0}{\sigma/\sqrt{n}} < z_{\alpha}\right\} \\ &= P_{\mu}\left\{\frac{\bar{X} - \mu}{\sigma/\sqrt{n}} < z_{\alpha} - \frac{\mu - \mu_0}{\sigma/\sqrt{n}}\right\} \\ &= \phi(z_{\alpha} - \lambda)\end{aligned}$$

其中, $\lambda = \frac{\mu - \mu_0}{\sigma/\sqrt{n}}$, σ 表示方差。

$\beta(\mu)$ 是单调递减连续函数, 当 $\mu \geq \mu_0 + \delta (\delta > 0)$ 时, 有 $\beta(\mu) \leq \beta(\mu_0 + \delta)$ 。

为使犯第 II 类错误的概率不超过给定的 β , 需要 $\beta(\mu_0 + \delta) = \phi(z_{\alpha} - \sqrt{n}\delta/\sigma) \leq \beta$,

即 n 满足 $z_{\alpha} - \sqrt{n}\delta/\sigma \leq -z_{\beta}$ 。

所以, 只需 $\sqrt{n} \geq \frac{(z_{\alpha} + z_{\beta})\sigma}{\delta}$ 。

5.1.2 样本容量与临界次品数量

1. 假设检验

基于置信水平, 给出假设:

$$H_0 : p \geq p_0 = 0.1, H_1 : p < p_0 = 0.1,$$

对于第一种方案: 在 95% 的置信度下认定零配件次品率超过标称值, 即拒绝备择假设, 从而不轻易接受这批零配件; 对于第二种方案: 在 90% 的置信度下认定零配件次品率不超过标称值, 即拒绝零假设, 从而不轻易拒绝这批零配件。

2. 样本容量的选取

已知 0-1 分布的均值和方差分别为 $\mu = p, \sigma^2 = p(1-p)$, 设 $X_1, X_2, \dots, X_n$ 是一个样本, 因样本容量 n 较大, 由中心极限定理知

$$\frac{\sum_{i=1}^n X_i - np}{\sqrt{p(1-p)}} = \frac{n\bar{X} - np}{\sqrt{p(1-p)}},$$

其近似地服从 $N(0, 1)$ 分布。

于是有

$$P\{-z_{\alpha/2} < \frac{n\bar{X} - np}{\sqrt{p(1-p)}} < z_{\alpha/2}\} \approx 1 - \alpha。$$

而

$$-z_{\alpha/2} < \frac{n\bar{X} - np}{\sqrt{p(1-p)}} < z_{\alpha/2} \Leftrightarrow (n + z_{\alpha/2}^2)p^2 - (2n\bar{X} + z_{\alpha/2}^2)p + n\bar{X}^2 < 0,$$

记

$$\begin{aligned}
 p_{min} &= \frac{1}{2a}(-b - \sqrt{b^2 - 4ac}) = \hat{p} - z_{\alpha/2} \sqrt{\frac{\hat{p}(1 - \hat{p})}{n}} \\
 p_{max} &= \frac{1}{2a}(-b + \sqrt{b^2 - 4ac}) = \hat{p} + z_{\alpha/2} \sqrt{\frac{\hat{p}(1 - \hat{p})}{n}} \\
 a &= n + z_{\alpha/2}^2 \\
 b &= -(2n\bar{X} + z_{\alpha/2}^2) \\
 c &= n\bar{X}^2,
 \end{aligned}$$

于是得到 p 的一个近似的置信水平为 $1 - \alpha$ 的置信区间 (p_{min}, p_{max}) ，其应处于 0.1 附近。

根据 z 检验的 OC 函数，能够得到满足显著性水平 α 及 β 的样本容量 n 的范围：

$$\sqrt{n} \geq \frac{(z_{\alpha/2} + z_{\beta})\sigma}{\delta} - > \sqrt{n} \geq \max \frac{(z_{\alpha/2} + z_{\beta})\sigma}{\delta}$$

其中，对于 n 重伯努利试验来说， $\sigma = \sqrt{p(1 - p)}$ 。为使抽样检测次数尽可能少，样本容量也应该选取最小值，即

$$\min n = \min \max \frac{z_{(\alpha/2 + z_{\beta})}^2 p(1 - p)}{\delta^2}$$

由于 $p \in (p_{min}, p_{max})$ ，且该区间位于二次函数 $f(p) = p(1 - p)$ 对称轴的左侧，当 $p = p_{max}$ 时取得 $\min n = \min \frac{z_{(\alpha/2 + z_{\beta})}^2 p_{max}(1 - p_{max})}{\delta^2}$ ，即 $n = \frac{z_{(\alpha/2 + z_{\beta})}^2 p_{max}(1 - p_{max})}{\delta^2}$ 。

3. 临界次品数量求解

由 n 重伯努利分布累积函数，可以求解临界次品数量。

对于方案一，令 $F_1(k) = \sum_{i=0}^k p_0^k (1 - p_0)^{n-k} = 0.95$ ，解得 $k = k_1$ ， k_1 为给定样本容量下最大容忍次品数，若检测到的次品数等于 k_1 ，则停止检测，认为超标，从而拒绝这批零配件。

对于方案二，令 $F_2(k) = \sum_{i=0}^k p_0^{n-k} (1 - p_0)^k = 0.90$ ，解得 $k = k_2$ ， k_2 为给定样本容量下最小接受良品数，若检测到的良品数等于 k_2 ，则停止检测，认为达标，从而接受这批零配件。

5.1.3 序列迭代方程

1. 方程构造

在实际样本的抽样过程中，每一次抽样的结果都会影响后续抽样的概率 $\hat{p}$ ，并造成样本容量 n 以及实际次品率置信区间上限 p_{max} 的动态变化，其即时值以下标 m 表示。

(a) 即时估计概率 $\hat{p}$ ：

$$\hat{p}_m = \frac{1}{m} \sum_{i=1}^m X_i = \frac{m-1}{m} \hat{p}_{m-1} + \frac{X_m}{m}, \text{ 记 } \hat{p}_0 = 0.$$

(b) 样本容量 n :

$$n = \frac{z_{(\alpha/2+z\beta)}^2 p_{max}(1-p_{max})}{\delta^2}$$

$$n_m = \frac{z_{(\alpha/2+z\beta)}^2 p_{max_{m-1}}(1-p_{max_{m-1}})}{\delta^2}$$

(c) 置信区间上限 p_{max} :

$$p_{max} = \frac{1}{2a}(-b + \sqrt{b^2 - 4ac}) = \hat{p} + z_{\alpha/2} \sqrt{\frac{\hat{p}(1-\hat{p})}{n}}$$

$$p_{max_m} = \hat{p}_m + z_{\alpha/2} \sqrt{\frac{\hat{p}_m(1-\hat{p}_m)}{n_m}}$$

(d) 临界次品数量 k_m :

$$F(k_m) = \sum_{i=0}^{k_m} p_0^{k_m} (1-p_0)^{n-k_m} = \alpha$$

对以上方程进行多次蒙特卡洛模拟，如以下系列图图??所示，注意到样本容量 n 与即时估计概率 $\hat{p}$ 拥有相似的趋势。当样本容量 n 稳定时，概率 $\hat{p}$ 同样保持稳定，保证了最终解能够满足置信水平并对概率 $\hat{p}$ 有较为精确的估计。

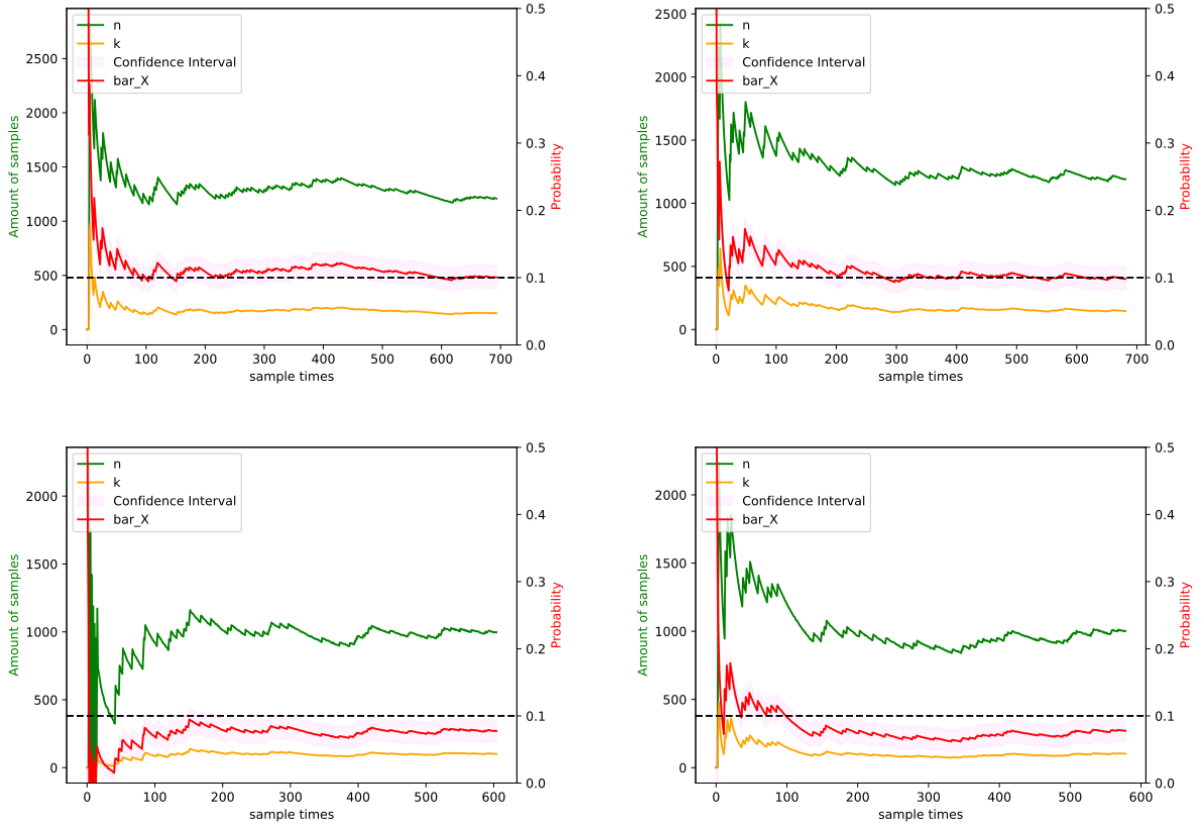


图 4 实例

此外，通过多次采样，我们发现将第 m 次概率估计值 $\hat{p}_m$ 代入到分布累积函数中，并对 $\hat{p}_m$ 进行如下修正：

$$\begin{aligned} err &= k_m - \hat{p}_m \times n_m \\ \hat{p}'_m &= \hat{p}_m - 0.75 \times \frac{err}{n_m} \\ \hat{p}'_m &= \min(\max(\hat{p}_m, 0), 1) \end{aligned}$$

迭代效果将更加快速，结果也更为精确。

2. 方程求解

- (a) 给定 $\beta = 0.10, \delta = 0.05, \text{MAXITER} = 1000$;
- (b) for i in range(MAXITER)
 - i. 初始化 $n_0 = 1, p_0 = 0$;
 - ii. while not stop
 - A. if n 收敛, stop
 - B. else 依次计算 $\hat{p}_m, n_m, p_{max_m}$;
 - iii. 保存当前样本容量 n_m ;
- (c) 对所有 n_m 求平均，得到最优样本容量;
- (d) 依据 n 重伯努利试验分布累积函数，计算临界 k 值。

5.1.4 问题解决：抽样检测方案

定义在置信水平 $1 - \alpha$ 下的关于样本容量 n 的次品数量表达式 $K_{1-\alpha}(n)$ ，其满足：

$$F(K(n), n, p) = 1 - \alpha,$$

对于情形一，取样本容量 n 为 470 左右，在检测过程中，当检测到的次品数量为 $K_{1-0.05}(n)$ 时停止检验，认为零配件次品率超过标称值，拒收这批零配件。如下图 ?? 所示，随着实际次品率的增加，需要更多的样本容量以满足置信水平。

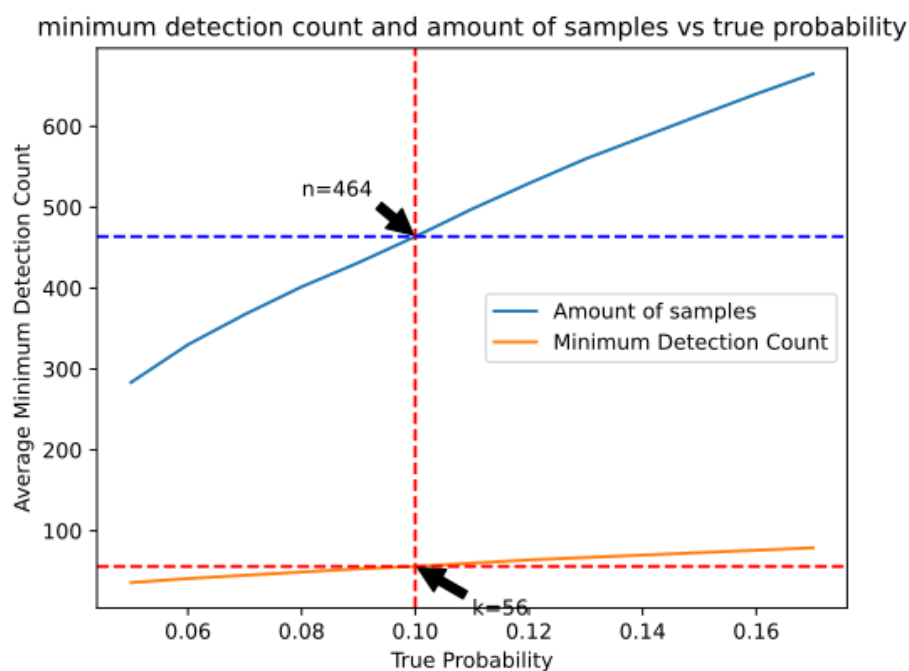


图 5 情形一：实际次品率的变化对抽样检测方案的影响

对于情形二，取样本容量 n 为 370 左右，在检测过程中，当检测到的次品数量为 $K_{1-0.10}(n)$ 时停止检验，认为零配件次品率不超过标称值，接受这批零配件。如下图??所示，随着实际次品率的增加，同样需要更多的样本容量以满足置信水平。

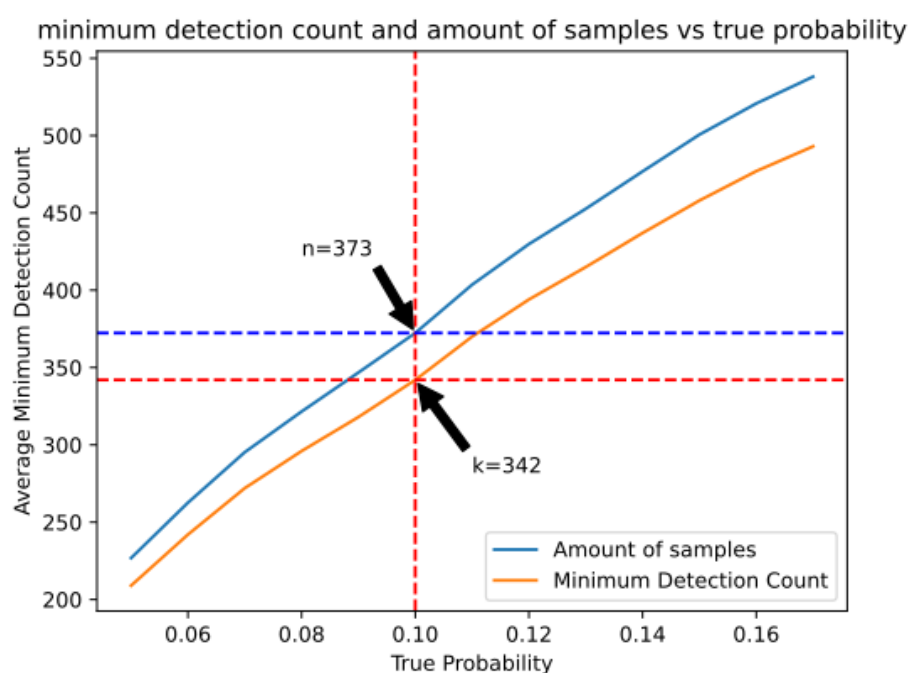


图 6 情形二：实际次品率的变化对抽样检测方案的影响

两种情形中，尽管当实际次品率小于 0.1 时，直觉上需要更多的检测次数，但由于其样本容量不是很大，导致其对应的良品数临界值较小，所以实际检测过程中更可能以较少的抽样次数结束抽样。

5.2 问题二 基于蒙特卡洛思想的模拟优化实验

5.2.1 蒙特卡洛算法 (Monte Carlo Method)

1. 理论基础——大数定律

- 伯努利大数定理： N 次独立重复实验，事件 A 发生的频率为 $\frac{n_A}{N}$ ，其随着试验次数的增大，依概率收敛为事件发生的概率 p_A ；
- 辛钦大数定理：从独立同分布、具有数学期望 μ 的随机变量序列 $x_1, x_2, x_3, \dots, x_n$ 中，抽取样本集合 X ，样本均值随样本量 n 的增大收敛为总体均值；
- 切比雪夫大数定理：独立的随机变量序列 $x_1, x_2, x_3, \dots, x_n$ ，期望为 $E(X_i) = \mu_i$ ，方差为 $D(X_i) = \sigma_i^2 < M$ ，抽取样本集合 X ，样本均值随样本量 n 的增大收敛为总体均值。

2. 蒙特卡洛算法

基于大数定理，蒙特卡洛算法通过大量随机样本的模拟，来估计一个问题的解，核心思想是随机采样。其具有随机性，导致结果具有不确定性，但在模拟样本足够多的情况下，可以收敛到近似解。由于进行大量样本的模拟，蒙特卡洛算法的计算量较大，但是可以很好地适用于无法解析求解的问题。

5.2.2 蒙特卡洛优化实验

由蒙特卡洛思想出发，本文设计了一种大样本实验模拟算法。算法以两种大量等量零配件开始，按照指定的次品率假定各个零配件的良次情况，在打乱后投入生产线。一个零配件将在生产线上流动，经历零配件检验、组装、成品检验、出售、调换、拆解等环节的决策过程。为了避免一个零配件反复流回生产线致使生产线无法完工，我们为每个零配件的被使用次数进行赋值，每流过一回生产线加一，当其超过最大的可接受使用次数，自动作废。在一种零配件耗尽后，统计各种决策组合的售出数目和总成本，获得纯利润，在比较后选择最优决策方案。

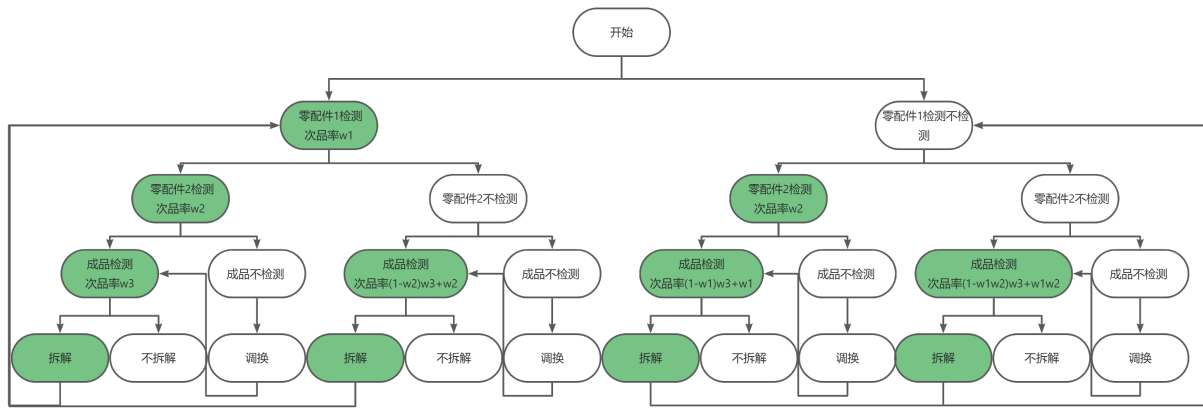


图 7 决策树

上图??给出了整个决策问题的树状图。以下给出蒙特卡洛优化实验的具体步骤：

1. 参数初始化：按照情况，选取对应的参数；
2. 决策编码：按照 0 为不进行、1 为进行对两个零配件是否检测、成品是否检测、次品是否拆解进行二进制编码，即 0000 到 1111 共 16 种决策方案；
3. 多重决策：每次取出一对零配件进入流水线，按某一决策方案进行生产，在过程中累计总成本：
 - (a) 零配件初始化：选取各零配件 10000 个作为样本，按照次品率给出良次情况，并打乱顺序。每个零配件初始的使用次数为 0；
 - (b) 检查使用次数：丢弃使用次数超过最大可接受次数的零配件，重新取出。对零配件的使用次数加一；
 - (c) 零配件检测：对决策方案选择检测点零配件进行检测，丢弃次品，重新取出；
 - (d) 成品检测：按照零配件检测情况和决策方案对成品进行检测；
 - (e) 拆解：按决策方案对次品进行拆解，拆解则继续使用当前零配件，不拆解则重新取出；
 - (f) 售出反馈：如果售出良品，增加售出数目；如果售出次品，按照上步中相同决策决定是否拆解；
4. 最优决策方案：比较由售出数目和总成本计算得到的纯利润，选择该情况的最优决策方案。

5.2.3 问题解决：各情况的最优决策方案

表 2 问题二情况

项目	情况 1	情况 2	情况 3	情况 4	情况 5	情况 6
零件 1 次品率	0.1	0.2	0.1	0.2	0.1	0.05
零件 1 检测成本	2	2	2	1	8	2
零件 2 次品率	0.1	0.2	0.1	0.2	0.2	0.05
零件 2 检测成本	3	3	3	1	1	3
成品次品率	0.1	0.2	0.1	0.2	0.1	0.05
成品检测成本	3	3	3	2	2	3
调换损失	6	6	30	30	10	10
拆解费用	5	5	5	5	5	40

根据以上方法，采用各零配件 10000 个，对上表??中 6 种情况对应的 16 种决策方案进行了对比，如以下系列图图 ??：

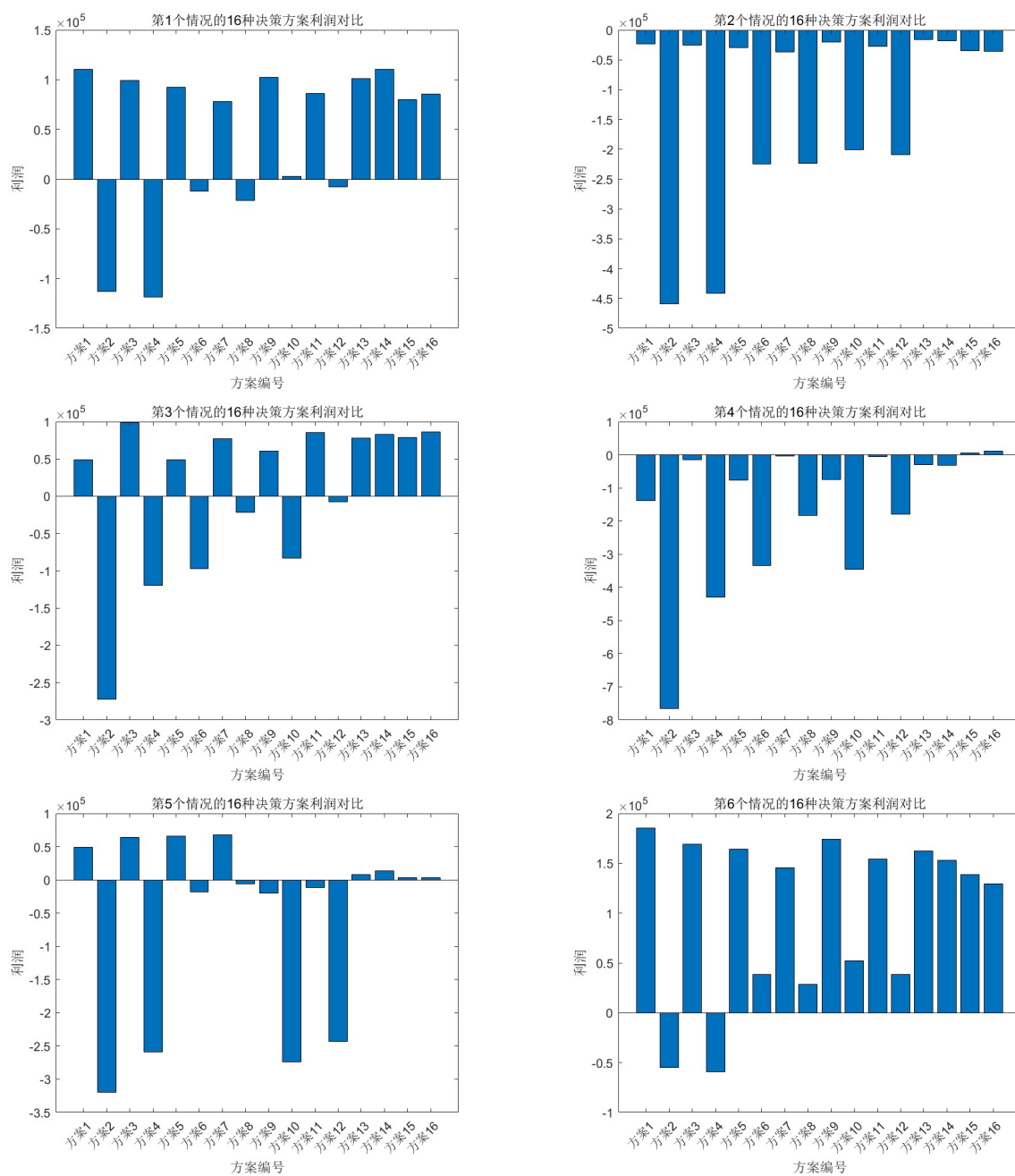


图 8 问题二各情况各决策方案对比

各情况的具体最优决策方案和预期最大利润如下表 ??:

表 3 问题二各情况最优决策方案和预期最大利润

情况	检测零配件 1	检测零配件 2	检测成品	拆解次品	预期最大利润（元）
1	1	1	0	1	110783
2	1	1	0	0	-15870
3	0	0	1	0	98856
4	1	1	1	1	11544
5	0	1	1	0	68186
6	0	0	0	0	185422

5.2.4 定性分析验证

为保证蒙特卡洛优化实验的结果正确，在无法进行实际生产过程的情况下，进行定性分析 (以情况 1 和情况 2 为基准)：

情况 3 点调换损失比情况 1 高，所以更需要保证出售的成品是良品。

情况 4 具有高次品率和低检测成本，最适合全面检测，也能保证拆解的利益。

情况 5 则是次品率低的零配件 1 检测成本高，次品率高的零配件 2 检测成本低，因此只检测后者的效益较高。

情况 6 的次品率最低，因此不进行检测会节省大量成本。

5.3 问题三 模型一般化

5.3.1 模型复杂度分析

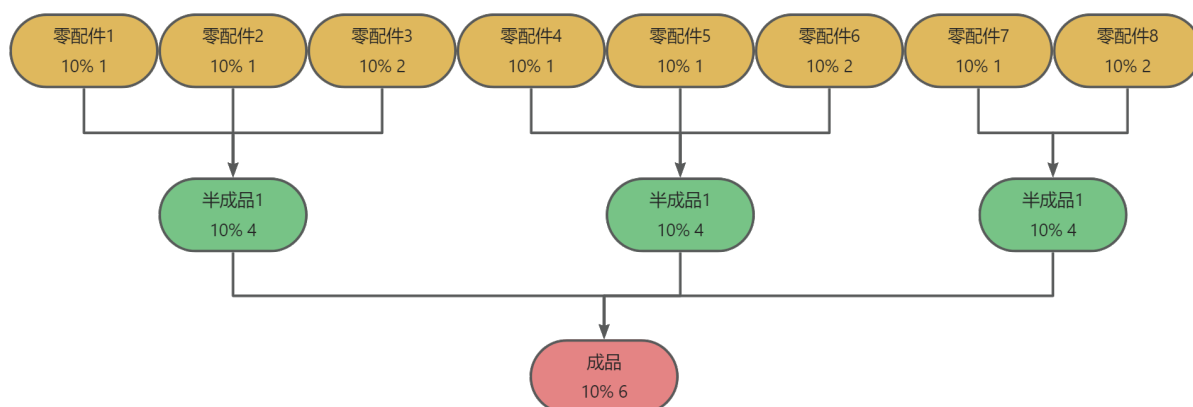


图 9 问题三步骤图

1. 决策组合遍历

代码通过嵌套的循环遍历所有可能的生产决策组合：检测零件 256 种组合，检测半成品 8 种组合，拆解半成品 8 种组合，检测成品 2 种组合，拆解成品 2 种组合，共 65536 种组合。然而， $O(65536) = O(1)$ ，这种复杂度是一个固定常量。

2. 生产过程模拟

生产过程中的主要复杂度来源于生产成品的数量，其与投入的零配件数相关。生产过程中有 $O(\text{numparts})$ 次模拟，每次模拟要逐个处理 8 种零件复杂度是 $O(8)$ ，生产过程的总复杂度是二者相乘。

3. 总体分析

根据以上分析可知，造成代码复杂度的主要因素是生产成品的数量。

5.3.2 问题解决：复杂问题求解

鉴于模型运算量较大，以下仅列举选取 1000 个各零配件的样本进行决策优化的结果：

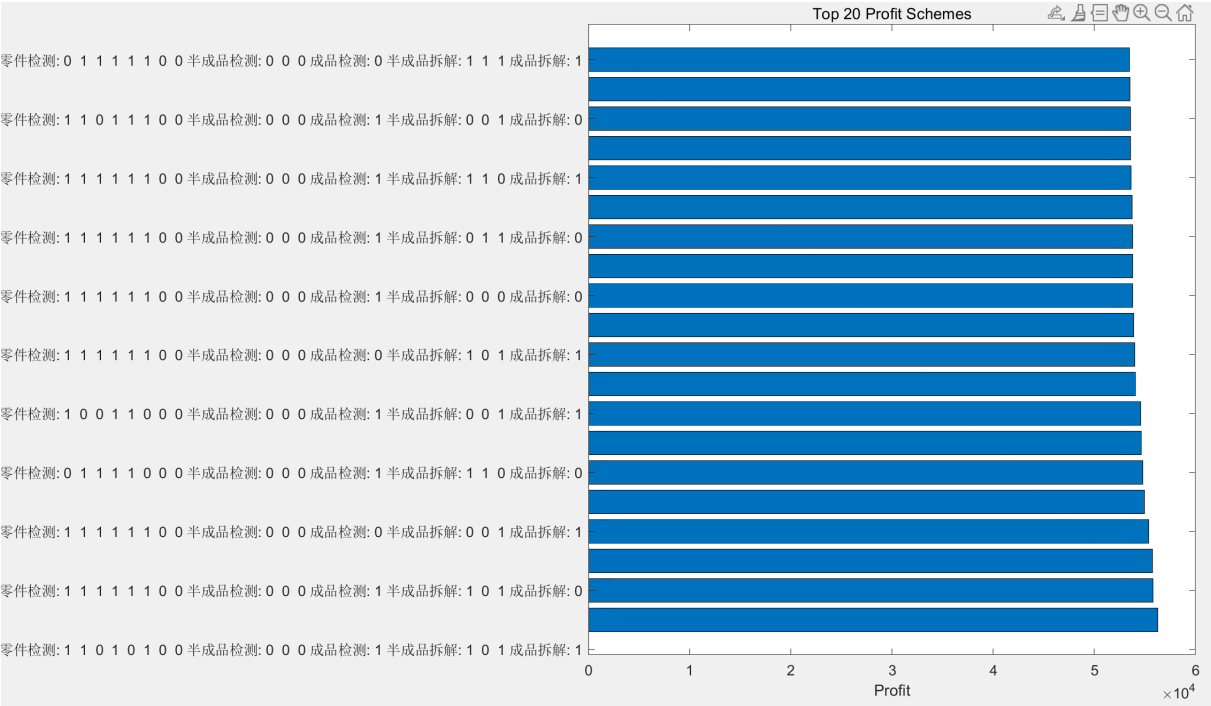


图 10 问题三 1000 次前 20 名方案

其中，决策方案二进制编码为 1101010000010111（零配件检测、半成品检测、半成品拆解、成品检测、成品拆解）的决策方案得到了最大的利润，为 56280 元。

5.3.3 简化模型的一般化

为使模型能应对一道工序、一个零配件的一般化问题，可以对模型进行适当简化，使其仅弱化利润的精确预测，保障决策方案的精确选择。模型可以从以下方面进行简化：

1. 适当减少投入的零配件数；
2. 将整个决策过程拆解为多个子决策过程，拆解点选择两个决策步骤间相互影响小的；
3. 对部分决策步骤提前进行人为的定性判断。

在实际生产生活中，企业、机构在实际生产前可使用更强大算力的计算机模拟投产零配件更多、规模更庞大的情况。

5.4 问题四 基于贝叶斯推断的实时模型

5.4.1 贝叶斯推断原理

1. 贝叶斯推断的原理与形式化描述

贝叶斯推理根据两个前因式——先验概率和由观测数据的统计模型得出的似然函数的结果来得到后验概率，计算依据是贝叶斯公式：

$$P(H|E) = \frac{P(E|H) \cdot P(H)}{P(E)} = \frac{P(E|H) \cdot P(H)}{P(E|H)P(H) + P(E|\bar{H})P(\bar{H})} = \frac{1}{1 + (\frac{1}{P(H)} - 1) \frac{P(E|\bar{H})}{P(E|H)}}$$

其中：

- (a) H 代表一组假设，相互竞争，概率受数据影响，问题在于决定最可能的假设；
- (b) $P(H)$ 代表先验概率，是在观测当前数据 E 之前，对假设 H 的概率估计；
- (c) E 代表未被用于计算先验概率的新数据；
- (d) $P(H|E)$ 代表后验概率（关于假设 H 的函数），其将 H 作用到 E ，表示在当前观测到的证据下，某个假设发生的概率有多大；
- (e) $P(E|H)$ 代表似然函数（关于证据 E 的函数），即假设 H 的前提下观测到 E 的概率，其能体现当前证据与给定假设的一致性；
- (f) $P(E)$ 代表边际似然函数（模型证据），其对所有被考虑到的可能的假设都相同，不会影响各个假设间的相对概率。

贝叶斯推理的核心为：

$$p(\theta|X, \alpha) = \frac{p(\theta, X, \alpha)}{p(X, \alpha)} = \frac{p(X|\theta, \alpha)p(\theta, \alpha)}{p(X|\alpha)p(\alpha)} = \frac{P(X|\theta, \alpha)p(\theta|\alpha)}{p(X|\alpha)} \propto p(X|\theta, \alpha)p(\theta|\alpha)$$

其中， x 表示一个数据点， θ 为其对应的分布参数， $x \sim p(x|\theta)$ ， α 为参数分布的超参数， $\theta \sim p(\theta|\alpha)$ ， $X = \{x_1, x_2, \dots, x_n\}$ 表示由 n 个观测的数据点构成的采样集，而 $\tilde{x}$ 表示一个新的数据点。

2. beta 分布

Beta 分布一般用于建模伯努利试验事件成功的概率的概率分布，是一种连续性概率密度分布，表示为 $x \sim \text{Beta}(\alpha, \beta)$ ，参数 α, β 为形状参数。其概率密度函数为

$$p(\theta) = \frac{\Gamma(\alpha+\beta)}{\Gamma(\alpha)\Gamma(\beta)} \theta^{\alpha-1} (1-\theta)^{\beta-1}$$

参数 α 、 β 的不同组合，能够得到类似于 U 型分布、正态分布、均匀分布、指数分布等众多不同分布的形状，如下图??所示具有很强的通用性。

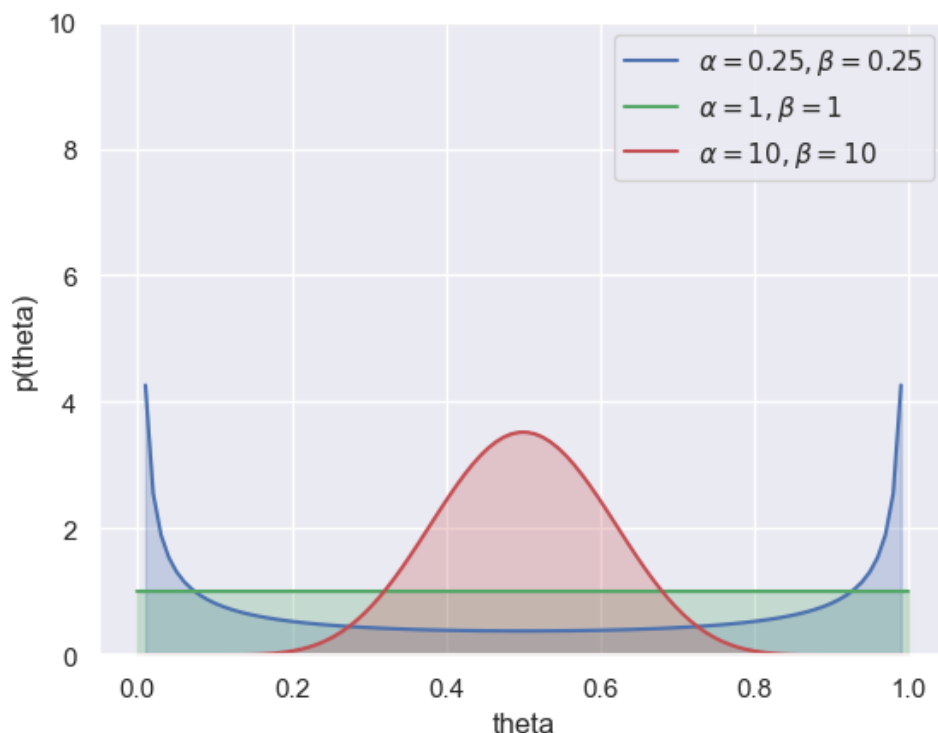


图 11 参数对分布形状的影响

5.4.2 问题解决：实施贝叶斯推理实时修正

1. 基于贝叶斯推理的生产实时修正

对于一批零配件、半成品或者成品，一旦其进入检测环节，就会对 $\hat{p}$ 产生影响。利用上一步估计值 $\hat{p}$ 替代原始模型中的实际次品率 p 求解，并在求解过程中利用贝叶斯推断进行修正。

假设实际次品率 θ 满足 $\theta \sim \text{Beta}(\alpha, \beta)$ ，则 $\hat{p}$ 是该分布出现概率最大的随机变量。统计进入检测环节的物品总数 n 、检测到的次品数 k ，基于初始估计值，有 $p(k|\theta) =$

$C_n^k \theta(1-\theta)^{n-k}$ 。根据贝叶斯推理 $f(\theta|k) \propto f(\theta)p(k|\theta)$ 计算后验概率：

$$\begin{aligned} f(\theta) & \frac{\Gamma(\alpha+\beta)}{\Gamma(\alpha)\Gamma(\beta)} \theta^{\alpha-1}(1-\theta)^{\beta-1} \\ p(k|\theta) & = C_n^k \theta(1-\theta)^{n-k} \\ f(\theta|k) & \propto \frac{\Gamma(\alpha+\beta)}{\Gamma(\alpha)\Gamma(\beta)} \theta^{\alpha-1}(1-\theta)^{\beta-1} C_n^k \theta^k(1-\theta)^{n-k} \end{aligned}$$

通过进一步化简，将与未知参数 θ 无关的项合并到归一化项中，有

$$f(\theta|k) \propto \theta^{\alpha-1}(1-\theta)^{\beta-1} \theta^k(1-\theta)^{n-k} = \theta^{\alpha+k-1}(1-\theta)^{\beta+n-k-1}$$

其在添加归一化项后，在 θ 定义域内积分为 1。

此外，后验分布显然是 **beta** 分布，参数 $\alpha' = \alpha + k, \beta' = \beta + n - k$ 作为新 **beta** 分布的参数。

2. 具体算法步骤：

- (a) 随机初始化 **beta** 分布参数 α, β ;
- (b) 对于每一个决策方案的生产过程，依当前决策，在检测环节（如果检测）中，统计检测的零件/半成品/成品数记为 n ，其中次品数量记为 k ;
- (c) 更新 $\alpha = \alpha + k, \beta = \beta + n - k$;
- (d) 在当前 **beta** 分布函数下寻找满足 $\hat{p} = \operatorname{argmax}(f(\hat{p}|k))$ ，即为当前次品率估计值；
- (e) 将估计值替代原始问题中的真实值，并寻找当前最优决策；
- (f) 重复步骤 2-5。

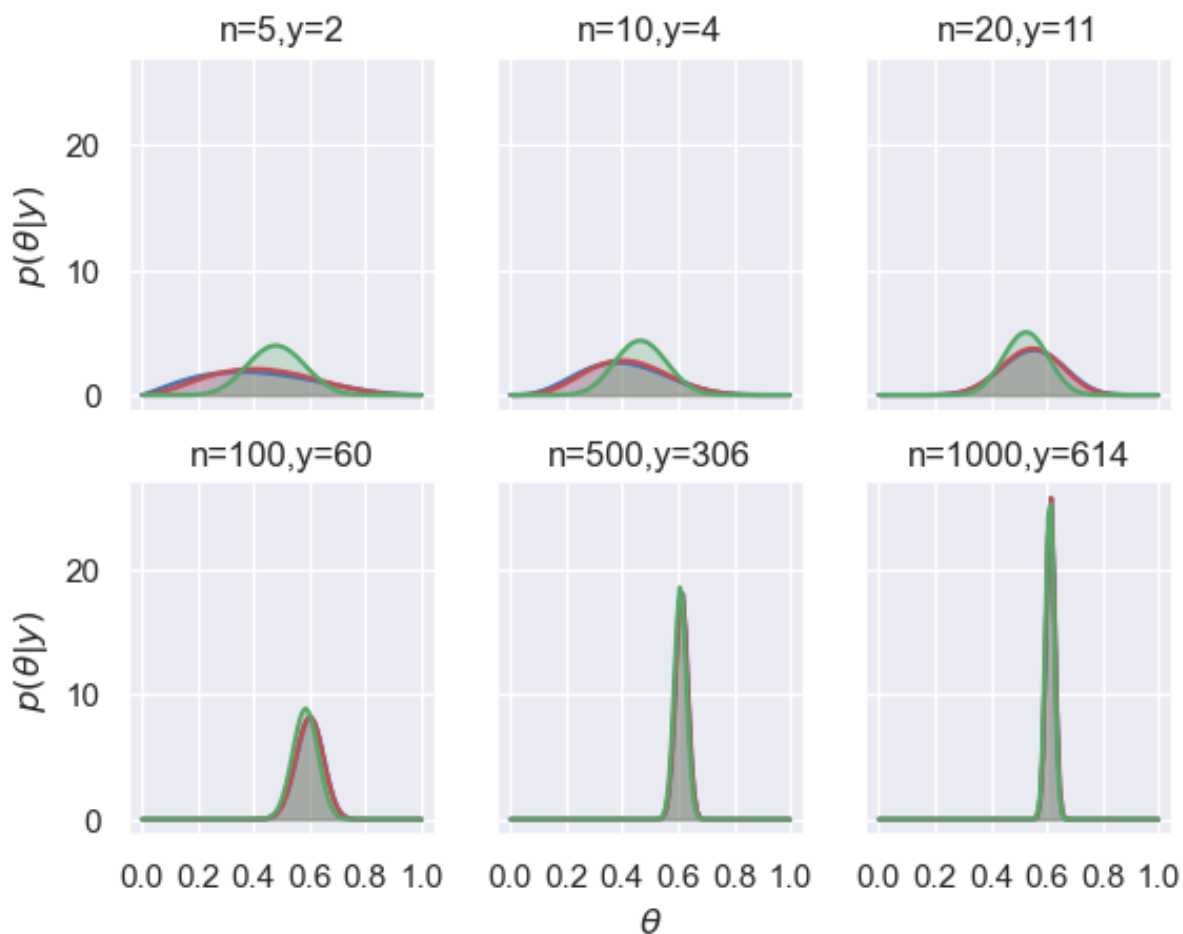


图 12 随着检测的进行，估计值将具有更高的确定性

3. 针对问题二、问题三的模拟本文给出了加入贝叶斯推断改进的模型关于问题二、问题三的解，由于维度过高，这里不进行完全展示，请见支撑文件中的 q4-2profit.mat、q4-2solution.mat、q4-3profit.ma。

六、模型评价与推广

6.1 模型优点

1. 本文将复杂实际模拟与简化模型评估相结合，寻求了精确度与便捷性的统一。由于实际模拟实验的存在，本模型在定量和定性两个层面的分析中都表现得十分牢靠。
2. 本文周全地考虑了问题中各种出现的情形，没有忽略和大规模简化，保证了模型结果与实际的高度统一。

6.2 模型特色与创新点

本文创新性地引入了两种方法，对本类生产决策问题的针对性很强。

1. 简化的序贯概率比检验：将判断是否可信的问题带入到抽样检测的动态过程中，可以极大减少检测需求的样本容量。通过进一步延伸，还可以在未知次品率标称值的情况下，给出符合可信度要求的最小的样本容量。
2. 基于蒙特卡洛算法的模拟优化实验：方法对生产问题进行大体量的模拟实验，直观真实，其模拟精度与样本容量正相关，且由于每次模拟实验都是独立的，可以并行进行。其使模型能够处理复杂的优化决策系统，且无需严格的数学模型，具有广泛的适用性。

6.3 模型缺点

1. 模型的各部分都批量进行重复实验，复杂度较高，计算量较大，对设备的并行能力提出了一定要求。
2. 基于蒙特卡洛算法的模拟优化实验虽然直观，但解释性差，且与蒙特卡洛算法一样具有随机性的问题。

6.4 模型推广

本文给出的多重决策模型广泛地适用于生产生活中的大量决策问题，具有极佳的推广性和普适性。尤其是在面对实际的生产问题时，模型并不受到标称值未知的影响，不但给出了只受置信度影响的最小检测数目计算的方法，还将其直接纳入到整个决策问题中去，一举多得，方便快捷，而且鲁棒性较强。

6.5 进一步研究需求

1. 虽然蒙特卡洛算法提供了基于随机采样思想的模拟实验，但其本身并不能作为证明依据，现实中的实际应用验证也是需要的。
2. 模型在不同的情况下都求解了检测所需的最小的样本容量，但在实际问题中，由于待检测物品的数目不知，这部分检测的比重难以衡量。对于数量极多的物品，抽样检测几百乃至几千个的成本可以忽略；而对于相近数量级的样品，这部分费用将产生重要影响。为了深化这部分改变对模型效果的影响，需要更多更全面的问题情形。
3. 模型只考虑了利润这一个量化评判标准，没有对时间成本、人工成本、生产线运行成本等实际问题中应考虑的多重量化标准进行评判，这是由于问题提出的简化，不是模型自身的问题。如果有足够的实际运营经验和数据，可以利用模拟退火、辅助常量等方法，加入这些评判依据，使模型更加全面。

参考文献

- [1] 左正康, 孙欢, 王昌晶, 等. 命令式动态规划类算法程序推导及机械化验证 [J]. 软件学报, 2024, 35(09): 4218-4241. DOI: 10.13328/j.cnki.jos.007134.
- [2] 刘小虎, 李生. 决策树的优化算法 [J]. 软件学报, 1998, (10): 78-81.
- [3] 连可, 陈世杰, 周建明, 等. 基于遗传算法的 SVM 多分类决策树优化算法研究 [J]. 控制与决策, 2009, 24(01): 7-12. DOI: 10.13195/j.cd.2009.01.9.liank.012.

附录 A 支撑文件

- montecarlo.py: 问题一代码 1, 测试简化收敛性;
- findBestSolution.py: 问题一代码 2, 求解样本容量;
- q2.m: 问题二代码;
- q3.m: 问题三代码;
- betaDistribute.py: 问题四代码 1, beta 分布概率密度图;
- betapara.py: 问题四代码 2, beta 参数对形状的影响;
- q42.m: 问题四针对问题二的代码;
- q43.m: 问题四针对问题三的代码;
- q4-2profit.mat: 问题四针对问题二的不同概率所得最大利润;
- q4-2solution.mat: 问题四针对问题二的不同概率所得最优决策;
- q4-3profit.mat: 问题四针对问题三的不同概率所得最大利润;

附录 B montecarlo

```
import numpy as np
import matplotlib.pyplot as plt
import scipy.stats as stats
from math import sqrt, ceil
from scipy.stats import binom

def critical_k(n, alpha, p=0.1):
    p_result = binom.pmf(np.arange(n+1), n, p) #计算所有k值的概率
    p_sum = np.cumsum(p_result) #计算累积概率
    return np.searchsorted(p_sum, 1-alpha, side='right') - 1

alpha = 0.05 #1-a = 0.95
beta = 0.10 #1-b = 0.90
delta = 0.03 #概率值容忍误差
n_init = 1 #初始样本量
#计算置信区间
z_alpha = stats.norm.ppf(1-alpha/2)
z_beta = stats.norm.ppf(1-beta)
#计算系数
u = (z_alpha + z_beta)**2 / (delta**2)
#相关参数
p_0 = 0.1
sight = 10
MAXITER = 1000
#初始化参数
X = [] #样本序列
```

```

X_sum = []          #样本和序列
bar_X = []          #样本均值序列
p_max = []          #最大置信区间
p_min = []          #最小置信区间
n = [n_init]        #样本量序列
k = []              #置信区间上界
count = 0            #迭代次数
#迭代
while 1:
    #如果已经迭代足够次数且n的变化趋于稳定，则跳出循环
    if count > sight and (np.abs(n[-1] - np.mean(n[-sight:]))) <= 5 and np.var(n[-sight:]) <= 10:
        print(f"n: {n[-1]}, d: {n[-1]-np.mean(n[-sight:])}, var: {np.var(n[-sight:])}, k: {critical_k(n[-1], alpha)}")
        break
    #更新计数器
    count += 1
    #生成一个二项分布的随机样本
    X.append(np.random.binomial(1, p_0))
    #更新参数
    X_sum.append(sum(X))
    bar_X.append(sum(X)/len(X))
    #print(bar_X[-1]) #用于调试
    #更新样本量
    if count > 1:
        n.append(ceil(max(count, u * p_max[-1] * (1-p_max[-1]))))
    #最大迭代次数限制
    if count > MAXITER:
        print("Max iteration reached")
        break
    #更新临界次品数目
    k.append(critical_k(n[-1], alpha, bar_X[-1]))
    #修正项计算并修正参数
    err = k[-1] - bar_X[-1] * n[-1]
    bar_X[-1] -= 0.75 * err / n[-1]
    bar_X[-1] = min(max(bar_X[-1], 0), 1) #限制置信区间在[0,1]范围内
    #计算置信区间
    p_max.append(min(bar_X[-1]+z_alpha * sqrt(bar_X[-1]*(1-bar_X[-1])/n[-1]), 1))
    p_min.append(max(bar_X[-1]-z_alpha * sqrt(bar_X[-1]*(1-bar_X[-1])/n[-1]), 0))

print(f"bar_X: {bar_X[-1]}, p_min: {p_min[-1]}, p_max: {p_max[-1]}, n: {n[-1]}, k: {k[-1]}, count: {count}")
#绘图，使用两个比例尺
fig, ax1 = plt.subplots()
ax1.plot(n, label='n', color='green')
ax1.plot(k, label='k', color='orange')
ax1.set_xlabel('sample times')

```

```

ax1.set_ylabel('Amount of samples', color='green')
ax1.tick_params(axis='y', labelcolor='black')
ax2 = ax1.twinx()
#置信区间绘制
#ax2.plot(p_max, label='p_max', color='purple', linestyle='--')
#ax2.plot(p_min, label='p_min', color='orange', linestyle='--')
#以如下形式展现
ax2.fill_between(range(len(p_max)), p_min, p_max, color='magenta', alpha=alpha,
                 label='Confidence Interval')
ax2.plot(bar_X, label='bar_X', color='red')
ax2.set_ylabel('Probability', color='red')
ax2.tick_params(axis='y', labelcolor='black')
ax2.set_ylim(0, 0.5)
plt.axhline(y=p_0, linestyle='--', color='black')
lines1, labels1 = ax1.get_legend_handles_labels()
lines2, labels2 = ax2.get_legend_handles_labels()
ax1.legend(lines1 + lines2, labels1 + labels2, loc='upper left')

```

附录 C findBestSolution

```

import numpy as np
import matplotlib.pyplot as plt
import scipy.stats as stats
from math import sqrt, ceil
from scipy.stats import binom

#计算给定置信水平下，二项分布需要达到的临界k值
def critical_k(n, alpha, p=0.1):
    p_result = binom.pmf(np.arange(n+1), n, p) #计算从0到n的所有k值的二项分布概率
    p_sum = np.cumsum(p_result) #计算累积概率
    return np.searchsorted(p_sum, 1-alpha, side='right') - 1
    #找到累积概率大于或等于1-alpha的最小k值

#找到最优样本量n的迭代过程
def findBestSolution(alpha, beta, delta, n_init, MAXITER, p=0.1):
    #计算正态分布的分位数
    z_alpha = stats.norm.ppf(1-alpha/2)
    z_beta = stats.norm.ppf(1-beta)
    #计算样本量n的公式中的u
    u = (z_alpha + z_beta)**2 / (delta**2)
    #初始化存储结果的列表
    ave = [] #平均迭代次数
    ave_n = [] #平均样本量
    result = [] #最优样本量的列表
    result_1 = [] #最优样本量对应的迭代次数的列表

```

```

#迭代过程
for i in range(MAXITER):
    #初始化变量
    X = []          #每次迭代的样本均值
    X_sum = []      #每次迭代的样本总和
    bar_X = []      #每次迭代的样本均值
    p_max = []      #每次迭代的样本均值+z_alpha*sqrt(bar_X*(1-bar_X)/n)的最大值
    p_min = []      #每次迭代的样本均值-z_alpha*sqrt(bar_X*(1-bar_X)/n)的最小值
    n = [n_init]    #每次迭代的样本量
    k = []          #每次迭代的临界k值
    count = 0       #迭代次数
    while 1:
        #如果已经迭代足够次数且n的变化趋于稳定，则跳出循环
        if count > 20 and (n[-1] - np.mean(n[-20:]) <= 10) and np.var(n[-20:]) < 10:
            result.append(n[-1])
            result_1.append(count)
            break
        #增加迭代次数
        count += 1
        #生成一个二项分布的随机样本
        #if random.random() < p:
        #    X.append(1)
        #else:
        #    X.append(0)
        X.append(np.random.binomial(1, 0.1))
        X_sum.append(sum(X))
        bar_X.append(sum(X)/len(X))
        #更新n的值
        if count > 1:
            n.append(ceil(max(count, u * p_max[-1] * (1-p_max[-1]))))
        #更新p_max和p_min
        p_max.append(min(bar_X[-1]+z_alpha * sqrt(bar_X[-1]*(1-bar_X[-1])/n[-1]), 1))
        p_min.append(max(bar_X[-1]-z_alpha * sqrt(bar_X[-1]*(1-bar_X[-1])/n[-1]), 0))
        #计算当前n下的临界k值
        k.append(critical_k(n[-1], alpha))
        #计算平均迭代次数和平均样本量
        ave.append(np.mean(result_1))
        ave_n.append(np.mean(result))
    #打印结果和最后一次迭代的详细信息
    print(np.mean(result))
    print(f"bar_X: {bar_X[-1]}, p_max: {p_max[-1]}, n: {n[-1]}, k: {k[-1]}, count: {count}")
    #绘制n和count的变化图
    plt.plot(n, label='n')
    plt.plot(range(count), label='count')
    plt.legend()
    plt.show()

```

```
#调用函数，测试不同的参数
#findBestSolution(alpha=0.05, beta=0.10, delta=0.05, n_init=1, MAXITER=500)
findBestSolution(alpha=0.10, beta=0.10, delta=0.05, n_init=1, MAXITER=10, p=1-0.1)
```

附录 D q2

%已知量

```
data=[0.1 0.2 0.1 0.2 0.1 0.05; %零配件1次品率
      2 2 2 1 8 2 ; %零配件1检测成本
      0.1 0.2 0.1 0.2 0.2 0.05; %零配件2次品率
      3 3 3 1 1 3 ; %零配件2检测成本
      0.1 0.2 0.1 0.2 0.1 0.05; %成品次品率
      3 3 3 2 2 3 ; %成品检测成本
      6 6 30 30 10 10 ; %调换损失
      5 5 5 5 5 40 ]; %拆解费用
cost_part_1 = 4; %零配件1购买单价
cost_part_2 = 18; %零配件2的购买单价
cost_assembly = 6; %装配成本
price_product = 56; %市场售价
```

%六种情况

```
for cases =1:6
    %% 初始化变量
    defective_part_1 = data(1,cases); %零配件1次品率
    cost_check_part_1 = data(2,cases); %零配件1检测成本
    defective_part_2 = data(3,cases); %零配件2次品率
    cost_check_part_2 = data(4,cases); %零配件2检测成本
    defective_product = data(5,cases); %成品次品率
    cost_check_product = data(6,cases); %成品检测成本
    cost_replace = data(7,cases); %调换损失
    cost_disassemble = data(8,cases); %拆解费用
    num_parts = 10000; %模拟投入生产线总共的零配件数
    num_bad_1 = num_parts * defective_part_1; %次品零配件1
    num_good_1 = num_parts - num_bad_1; %良品零配件1
    num_bad_2 = num_parts * defective_part_2; %次品零配件2
    num_good_2 = num_parts - num_bad_2; %良品零配件2
    max_uses = 3; %零配件最多使用次数为3次
    profits = zeros(1, 16); %记录各方案的利润
    best_profit = -Inf; %最大利润
    best_scheme = []; %最优方案
    index = 1; %方案索引
    %% 遍历所有生产决策组合（16种组合，分别表示每个阶段的检测决策）
    for check_part_1 = [0, 1]
        for check_part_2 = [0, 1]
            for check_product = [0, 1]
```

```

for disassemble_product = [0, 1]
    %初始化仓库中的零配件和使用次数
    parts_1 = [ones(1, num_good_1), zeros(1, num_bad_1)];
    parts_2 = [ones(1, num_good_2), zeros(1, num_bad_2)];
    parts_1 = parts_1(randperm(num_parts)); %随机打乱
    parts_2 = parts_2(randperm(num_parts));
    uses_part_1 = zeros(1, num_parts); %初始化零配件的使用次数
    uses_part_2 = zeros(1, num_parts);
    profit = 0; %初始化利润
    total_sales = 0; %初始化成交数
    i = 1; %初始化零配件1索引
    j = 1; %初始化零配件2索引
    cost=0;
    %模拟生产过程，直到某种零配件用完
    while i <= length(parts_1) && j <= length(parts_2)
        cost = cost + cost_part_1 + cost_part_2;
        %获取当前取出的零配件1和零配件2
        part_1 = parts_1(i);
        part_2 = parts_2(j);
        %检查零配件的使用次数，超出使用次数则更换零配件
        if uses_part_1(i) >= max_uses
            i = i + 1;
            continue; %零配件使用超过上限，丢弃
        end
        if uses_part_2(j) >= max_uses
            j = j + 1;
            continue;
        end
        %增加使用次数
        uses_part_1(i) = uses_part_1(i) + 1;
        uses_part_2(j) = uses_part_2(j) + 1;
        %根据方案决定是否检测零配件1
        if check_part_1
            cost = cost + cost_check_part_1;
            if part_1 == 0 %次品
                i = i + 1;
                continue; %丢弃次品，取下一个零配件1
            end
        end
        %根据方案决定是否检测零配件2
        if check_part_2
            cost = cost + cost_check_part_2;
            if part_2 == 0
                j = j + 1;
                continue;
            end
        end
    end
end

```

```

%装配成品
cost = cost + cost_assembly;
product = part_1 == 1 && part_2 == 1; %判断零配件是否都合格
%即使零配件1和零配件2都是良品，成品也有10%的概率次品
if product
    if rand() <= defective_product
        product = 0; %模拟成品为次品的概率
    end
end
%成品检测
if check_product
    cost = cost + cost_check_product;
    if ~product
        %成品检测不合格
        if disassemble_product %决定是否拆解成品
            cost = cost + cost_disassemble;
            continue;
        else
            i = i + 1;
            j = j + 1;
            continue;
        end
    end
end
%客户收到好货，更新利润和销售数
if product
    total_sales = total_sales + 1;
    i = i + 1;
    j = j + 1;
end
%如果客户收到次品
if ~product
    cost = cost + cost_replace;
    if disassemble_product %决定是否拆解成品
        cost = cost + cost_disassemble;
        continue
    else
        i = i + 1;
        j = j + 1;
        continue
    end
end
end %循环到这里
profit = total_sales * price_product - cost;
profits(index) = profit;
index = index + 1;
%更新最优方案

```

```

        if profit > best_profit
            best_profit = profit;
            best_scheme = [check_part_1, check_part_2, check_product,
                           disassemble_product];
        end
    end
end
end
end
end
%输出最优方案
fprintf('第%d个情况的最优决策方案: \n', cases);
fprintf('检测零配件1: %d\n', best_scheme(1));
fprintf('检测零配件2: %d\n', best_scheme(2));
fprintf('检测成品: %d\n', best_scheme(3));
fprintf('拆解次品: %d\n', best_scheme(4));
fprintf('最大利润: %.2f\n', best_profit);
%绘制对比图
figure;
bar(profits);
title(sprintf('第%d个情况的16种决策方案利润对比', cases));
xlabel('方案编号');
ylabel('利润');
xticks(1:16);
set(gca, 'XTickLabel', arrayfun(@(x) sprintf('方案%d', x), 1:16, 'UniformOutput', false));
end

```

附录 E q3

```

%% 输入已知量
cost_parts = [2, 8, 12, 2, 8, 12, 8, 12]; %八种零配件的采购成本
cost_check_parts = [1, 1, 2, 1, 1, 2, 1, 2]; %八种零配件的检测成本
defective_parts = [0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1]; %八种零配件的次品率
cost_assembly_half_products=[8, 8, 8]; %三个半成品的装配成本
cost_check_half_products=[4,4,4]; %三个半成品的检测成本
cost_disassemble_half_products = [6, 6, 6]; %三个半成品的拆解费用
defective_half_products = [0.1, 0.1, 0.1]; %三个半成品的次品率
cost_assembly_product = 8; %成品的装配成本
cost_check_product = 6; %成品的检测成本
cost_disassemble_product = 10; %成品的拆解费用
defective_product = 0.1; %成品的次品率
price_product = 200; %成品售价
cost_replace = 40; %不合格成品调换损失

%% 初始化变量
num_parts = 2000; %各零配件个数

```



```

num_bad_parts = round(num_parts.*defective_parts);
num_good_parts=num_parts-num_bad_parts;
schemes = {}; %方案
profits = []; %方案的利润

%% 遍历所有可能的生产决策组合
for check_parts_bin = 0:255 %零配件检测
    check_parts = num2cell(dec2bin(check_parts_bin, 8) - '0');
    for check_half_products_bin = 0:7 %半成品检测
        check_half_products = num2cell(dec2bin(check_half_products_bin, 3) - '0');
        for disassemble_half_products_bin = 0:7 %半成品拆解
            disassemble_half_products = num2cell(dec2bin(disassemble_half_products_bin, 3) -
                '0');
            for check_product = [0, 1] %成品检测
                for disassemble_product = [0, 1] %成品拆解
                    parts = cell(1, 8); %8种零配件
                    for k = 1:8
                        %每种零配件的良品和次品
                        parts{k} = [ones(1, num_good_parts(k)), zeros(1, num_bad_parts(k))];
                        %随机打乱顺序
                        parts{k} = parts{k}(randperm(num_parts));
                    end
                    uses_parts = zeros(1, 8); %每个零配件的使用次数
                    total_sales = 0; %总销售数量
                    cost = 0; %总成本
                    i = ones(1, 8); %各个零配件的索引
                    %模拟生产过程
                    num_products_to_assemble = floor(num_parts / 2); %最大成品数量
                    for product_num = 1:num_products_to_assemble
                        %检查是否还有足够的零配件继续生产
                        if any(i > num_parts)
                            break;
                        end
                        %获取当前零配件
                        parts_current = cellfun(@(x, idx) x(idx), parts, num2cell(i),
                            'UniformOutput', false);
                        %增加当前零配件的使用次数
                        for k = 1:8
                            uses_parts(k) = uses_parts(k) + 1;
                        end
                        discard = false;
                        for k = 1:8
                            %检查零配件
                            if check_parts{k} == 1 && parts_current{k} == 0
                                discard = true;
                                i(k) = i(k) + 1; %丢弃并跳过该零配件
                                break;
                            end
                        end
                    end
                end
            end
        end
    end
end

```

```

        end
    end
    if discard
        continue; %跳过本轮生产
    end
    %模拟装配和检测半成品
    half_products = [all([parts_current{1:3}]), all([parts_current{4:6}]),
        all([parts_current{5:6}])];
    %检测半成品
    for k = 1:3
        if check_half_products{k}
            if rand() <= defective_half_products(k)
                if disassemble_half_products{k}
                    cost = cost + cost_disassemble_half_products(k); %增加拆解费用
                end
                half_products(k) = false; %标记为次品
            end
        end
    end
    %装配成品
    if all(half_products)
        product = rand() > defective_product;
        cost = cost + cost_assembly_product; %增加装配成品的成本
        %检测成品
        if check_product && ~product
            cost = cost + cost_check_product; %增加检测成品的成本
            if disassemble_product
                cost = cost + cost_disassemble_product; %增加拆解成品的费用
            else
                cost = cost + cost_replace; %不拆解则增加替换成品的损失
            end
            continue; %跳过本轮生产
        end
        %记录销售数据
        if product
            total_sales = total_sales + 1; %成品合格，增加销售数量
        else
            cost = cost + cost_replace; %次品的替换损失
            if disassemble_product
                cost = cost + cost_disassemble_product; %增加拆解费用
            end
        end
    end
    i = i + 1; %更新零配件索引，继续生产
end
%计算利润

```

```

        profit = total_sales * price_product - cost;
        %保存当前方案和利润
        profits(end+1) = profit;
        schemes{end+1} = ['零配件检测: ', num2str([check_parts{:}])], ' 半成品检测: ',
            ...
                num2str([check_half_products{:}]), ' 成品检测: ',
                num2str(check_product), ...
                ' 半成品拆解: ', num2str([disassemble_half_products{:}]), '
                成品拆解: ', num2str(disassemble_product)];
    end
end
end
end
end
end

%% 计算前20个利润最大的方案
[sorted_profits, indices] = sort(profits, 'descend');
top_20_profits = sorted_profits(1:20);
top_20_schemes = schemes(indices(1:20));
%绘制柱状图
figure;
barh(top_20_profits);
set(gca, 'yticklabel', top_20_schemes); %y轴为方案名
xlabel('Profit');
ylabel('Scheme');
title('Top 20 Profit Schemes');

```

附录 F betaDistribute

```

import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import beta
import seaborn

seaborn.set()
params = [0.25, 1, 10]
x = np.linspace(0, 1, 100)
plt.plot(x, beta(0.25, 0.25).pdf(x), color='b', label='$\\alpha=0.25, \\beta=0.25$')
plt.fill_between(x, 0, beta(0.25, 0.25).pdf(x), color='b', alpha=0.25)
plt.plot(x, beta(1, 1).pdf(x), color='g', label='$\\alpha=1, \\beta=1$')
plt.fill_between(x, 0, beta(1, 1).pdf(x), color='g', alpha=0.25)
plt.plot(x, beta(10, 10).pdf(x), color='r', label='$\\alpha=10, \\beta=10$')
plt.fill_between(x, 0, beta(10, 10).pdf(x), color='r', alpha=0.25)
plt.gca().axes.set_ylim(0, 10)
plt.gca().axes.set_xlabel('theta')

```

```
plt.gca().axes. set_ylabel('p(theta)')
plt.legend()
plt.show()
```

附录 G betapara

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import beta
import seaborn

seaborn.set()
theta_real = 0.62
n_array = [5, 10, 20, 100, 500, 1000]
y_array = [2, 4, 11, 60, 306, 614]
beta_params = [(0.25, 0.25), (1, 1), (10, 10)]
x = np.linspace(0, 1, 100)
fig, ax = plt.subplots(2, 3, sharex=True, sharey=True)
for i in range(2):
    for j in range(3):
        n = n_array[3 * i + j]
        y = y_array[3 * i + j]
        for (a_prior, b_prior), c in zip(beta_params, ('b', 'r', 'g')):
            a_post = a_prior + y
            b_post = b_prior + n - y
            p_theta_given_y = beta.pdf(x, a_post, b_post)
            ax[i, j].plot(x, p_theta_given_y, c)
            ax[i, j].fill_between(x, 0, p_theta_given_y, color=c, alpha=0.25)
            ax[i, j].set_xticks([0, 0.2, 0.4, 0.6, 0.8, 1])
            ax[i, j].set_title('n={},y={}'.format(n, y))
ax[0, 0].set_ylabel('$p(\theta|y)$')
ax[1, 0].set_ylabel('$p(\theta|y)$')
ax[1, 1].set_xlabel('$\theta$')
plt.show()
```

附录 H q42

```
profit_p = zeros([6, 10, 10, 10]);
best_sol = zeros(6, 10, 10, 10, 4);
p = 0.01:0.1:0.95;
%已知量
for m = 1:10
```

```

for n = 1:10
    for q = 1:10
        data=[p(m) p(m) p(m) p(m) p(m) p(m); %零件1次品率
              2 2 2 1 8 2 ; %零件1检测成本
              p(n) p(n) p(n) p(n) p(n) p(n); %零件2次品率
              3 3 3 1 1 3 ; %零件2检测成本
              p(q) p(q) p(q) p(q) p(q) p(q); %成品次品率
              3 3 3 2 2 3 ; %成品检测成本
              6 6 30 30 10 10 ; %调换损失
              5 5 5 5 5 40 ]; %拆解费用
        cost_part_1 = 4; %零件1购买单价
        cost_part_2 = 18; %零件2的购买单价
        cost_assembly = 6; %装配成本
        price_product = 56; %市场售价
        fprintf('p_1=%.2f ', p(m));
        fprintf('p_2=%.2f ', p(n));
        fprintf('p_3=%.2f', p(q));
        %六种情况
        for cases =1:6
            %% 初始化变量
            defective_part_1 = data(1,cases); %零件1次品率
            cost_check_part_1 = data(2,cases); %零件1检测成本
            defective_part_2 = data(3,cases); %零件2次品率
            cost_check_part_2 = data(4,cases); %零件2检测成本
            defective_product = data(5,cases); %成品次品率
            cost_check_product = data(6,cases); %成品检测成本
            cost_replace = data(7,cases); %调换损失
            cost_disassemble = data(8,cases); %拆解费用
            num_parts = 10000; %模拟投入生产线总共的零件数
            num_bad_1 = num_parts * defective_part_1; %次品零件1
            num_bad_1 = ceil(num_bad_1);
            num_good_1 = num_parts - num_bad_1; %良品零件1
            num_good_1 = ceil(num_good_1);
            num_bad_2 = num_parts * defective_part_2; %次品零件2
            num_bad_2 = ceil(num_bad_2);
            num_good_2 = num_parts - num_bad_2; %良品零件2
            num_good_2 = floor(num_good_2);
            max_uses = 3; %零件最多使用次数为3次
            profits = zeros(1, 16); %记录各方案的利润
            best_profit = -Inf; %最大利润
            best_scheme = []; %最优方案
            index = 1; %方案索引
            %% 遍历所有生产决策组合（16种组合，分别表示每个阶段的检测决策）
            for check_part_1 = [0, 1]
                for check_part_2 = [0, 1]
                    for check_product = [0, 1]
                        for disassemble_product = [0, 1]

```

```

%初始化仓库中的零件和使用次数
parts_1 = [ones(1, num_good_1), zeros(1, num_bad_1)];
parts_2 = [ones(1, num_good_2), zeros(1, num_bad_2)];
parts_1 = parts_1(randperm(num_parts)); %随机打乱
parts_2 = parts_2(randperm(num_parts));
uses_part_1 = zeros(1, num_parts); %初始化零件的使用次数
uses_part_2 = zeros(1, num_parts);
profit = 0; %初始化利润
total_sales = 0; %初始化成交数
i = 1; %初始化零件1索引
j = 1; %初始化零件2索引
cost=0;
%模拟生产过程，直到某种零件用完
while i <= length(parts_1) && j <= length(parts_2)
    cost = cost + cost_part_1 + cost_part_2;
    %获取当前取出的零件1和零件2
    part_1 = parts_1(i);
    part_2 = parts_2(j);
    %检查零件的使用次数，超出使用次数则更换零件
    if uses_part_1(i) >= max_uses
        i = i + 1;
        continue; %零件使用超过上限，丢弃
    end
    if uses_part_2(j) >= max_uses
        j = j + 1;
        continue;
    end
    %增加使用次数
    uses_part_1(i) = uses_part_1(i) + 1;
    uses_part_2(j) = uses_part_2(j) + 1;
    %根据方案决定是否检测零件1
    if check_part_1
        cost = cost + cost_check_part_1;
        if part_1 == 0 %次品
            i = i + 1;
            continue; %丢弃次品，取下一个零件1
        end
    end
    %根据方案决定是否检测零件2
    if check_part_2
        cost = cost + cost_check_part_2;
        if part_2 == 0
            j = j + 1;
            continue;
        end
    end
end
%装配成品

```

```

cost = cost + cost_assembly;
product = part_1 == 1 && part_2 == 1; %判断零件是否都合格
%即使零件1和零件2都是良品，成品也有10%的概率次品
if product
    if rand() <= defective_product
        product = 0; %模拟成品为次品的概率
    end
end
%成品检测
if check_product
    cost = cost + cost_check_product;
    if ~product
        %成品检测不合格
        if disassemble_product %决定是否拆解成品
            cost = cost + cost_disassemble;
            continue;
        else
            i = i + 1;
            j = j + 1;
            continue;
        end
    end
end
%客户收到好货，更新利润和销售数
if product
    total_sales = total_sales + 1;
    i = i + 1;
    j = j + 1;
end
%如果客户收到次品
if ~product
    cost = cost + cost_replace;
    if disassemble_product %决定是否拆解成品
        cost = cost + cost_disassemble;
        continue
    else
        i = i + 1;
        j = j + 1;
        continue
    end
end
end %循环到这里
profit = total_sales * price_product - cost;
profits(index) = profit;
index = index + 1;
%更新最优方案
if profit > best_profit

```



```

num_good_parts=num_parts-num_bad_parts; %计算每种零件的良品数量
best_profit=-Inf; %初始化最大利润为负无穷
best_scheme=[]; %初始化保存最优方案的变量
%% 遍历所有可能的生产决策组合
for check_parts_bin = 0:255 %遍历所有检测零件的组合
    check_parts = num2cell(dec2bin(check_parts_bin, 8) - '0'); %生成8个零件的二进制决策
    for check_half_products_bin = 0:7 % 遍历半成品检测的组合
        check_half_products = num2cell(dec2bin(check_half_products_bin, 3) - '0');
        %生成3个半成品的二进制决策
        for disassemble_half_products_bin = 0:7 %遍历半成品拆解的组合
            disassemble_half_products = num2cell(dec2bin(disassemble_half_products_bin, 3) -
                '0'); %生成3个半成品的二进制决策
            for check_product = [0, 1] %是否检测成品
                for disassemble_product = [0, 1] %是否拆解成品
                    %初始化仓库中的零件和使用次数
                    parts = cell(1, 8); %创建一个包含8种零件的cell数组
                    for k = 1:8
                        %每种零件的良品和次品
                        parts{k} = [ones(1, num_good_parts(k)), zeros(1, num_bad_parts(k))];
                        %随机打乱顺序
                        parts{k} = parts{k}(randperm(num_parts));
                    end
                    uses_parts = zeros(1, 8); %使用数值数组记录每个零件的使用次数
                    total_sales = 0; %总销售数量
                    cost = 0; %总成本
                    i = ones(1, 8); %各个零件的索引
                    %模拟生产过程
                    %假设生产线能处理的最大成品数量为num_parts/2, 确保足够多的零件进行配对
                    num_products_to_assemble = floor(num_parts / 2);
                    for product_num = 1:num_products_to_assemble
                        %检查是否还有足够的零件继续生产
                        if any(i > num_parts)
                            break; %如果任意一个零件不足, 停止生产
                        end
                        %获取当前零件
                        parts_current = cellfun(@(x, idx) x(idx), parts, num2cell(i),
                            'UniformOutput', false);
                        %增加当前零件的使用次数
                        for k = 1:8
                            uses_parts(k) = uses_parts(k) + 1;
                        end
                        %根据方案检测零件
                        discard = false;
                        for k = 1:8
                            %检查零件是否需要检测, 以及是否为次品
                            if check_parts{k} == 1 && parts_current{k} == 0
                                discard = true;

```

```

        i(k) = i(k) + 1; %丢弃并跳过该零件
        break;
    end
end
if discard
    continue; %跳过本轮生产
end
%模拟装配和检测半成品
half_products = [all([parts_current{1:3}]), all([parts_current{4:6}]),
    all([parts_current{5:6}])];
half_products_costs = arrayfun(@(x) x +
    cost_assembly_half_products(x), 1:3);

%检测半成品
for k = 1:3
    if check_half_products{k}
        if rand() <= defective_half_products(k)
            if disassemble_half_products{k}
                cost = cost + cost_disassemble_half_products(k);
                %增加拆解费用
            end
            half_products(k) = false; %标记半成品为次品
        end
    end
end
%装配成品
if all(half_products)
    product = rand() > defective_product;
    %即使半成品良品，成品也可能是次品
    cost = cost + cost_assembly_product; %增加装配成品的成本
    %检测成品
    if check_product && ~product
        cost = cost + cost_check_product; %增加检测成品的成本
        if disassemble_product
            cost = cost + cost_disassemble_product; %增加拆解成品的费用
        else
            cost = cost + cost_replace; %不拆解则增加替换成品的损失
        end
        continue; %跳过本轮生产
    end

    %记录销售数据
    if product
        total_sales = total_sales + 1; %成品合格，增加销售数量
    else
        cost = cost + cost_replace; %次品的替换损失
        if disassemble_product

```

```

        cost = cost + cost_disassemble_product; %增加拆解费用
        continue
    end
end
end
%更新零件索引，继续生产
i = i + 1; %每次处理一个零件
end
%计算利润
profit = total_sales * price_product - cost;
%更新最优方案
if profit > best_profit
    best_profit = profit;
    best_scheme = [check_parts{:}, check_half_products{:},
        disassemble_half_products{:}, check_product, disassemble_product];
end
end
end
end
end
end
%输出最优方案
fprintf('最优方案: \n检测零件: %s\n检测半成品: %s\n拆解半成品: %s\n检测成品: %d\n拆解成品: %d\n最大利润:
    %.2f\n', ...
    num2str(best_scheme(1:8)), num2str(best_scheme(9:11)), num2str(best_scheme(12:14)),
    best_scheme(15), best_scheme(16), best_profit);
profit_p(ext) = best_profit;
end

```